

UNIVERSAL AI

GAME-PLAYING AGENT

Systems Architecture &
Feasibility Investigation

Research-only deliverable

Interface-first • Hierarchical • Verified • Safe

10 September 2026

Universal AI Game-Playing Agent

Systems Architecture and Feasibility Investigation

Date: 10 September 2026

Scope: Research and architecture only — no gameplay automation was implemented or tested.

Executive conclusion

A universal game-playing agent is technically feasible as a platform, but not as one universal policy that can reliably finish arbitrary commercial games. The feasible product is a universal orchestration shell with game-specific profiles, interface adapters, perception extractors, action mappings, skills, safety policies, and verification rules. Generality belongs in the control loop and data contracts; competence comes from adapters and learned or scripted skills.

The strongest practical design is hierarchical and multi-rate. A slow language model interprets goals, chooses skills, diagnoses novelty, and replans. Fast deterministic code and local models capture frames, extract telemetry, track objects/UI, execute controller trajectories, and enforce deadlines. A language model should never steer a car at 60 Hz, aim every frame, or time rhythm notes. It should request temporally extended skills such as follow-road, land-aircraft, navigate-to-marker, solve-menu, or dodge-pattern.

Two games in the collection offer especially strong legitimate structured interfaces: BeamNG.drive/BeamNG.tech through BeamNGpy and Microsoft Flight Simulator through SimConnect. Teardown offers a strong official Lua modding API, principally for controlled mod scenarios rather than transparent control of every stock campaign state. Most narrative action/horror titles require screen, audio, OCR, and ordinary user-level input. Online/live-service titles—especially Genshin Impact and Solo Leveling ARISE—should be disabled by default unless their current rules explicitly permit automation. The system must not inspect protected memory, inject into a process, evade anti-cheat, or automate competitive online play.

The right first proof of concept is not “play every game.” It is a benchmark harness plus three vertical slices: (1) BeamNG structured driving, (2) one offline menu-heavy strategy/sandbox game such as Plague Inc or WorldBox in screen mode, and (3) one offline third-person/platform title with a small objective. Success should be measured by verified subgoal completion, latency, recovery rate, and human interventions—not by impressive single-run videos.

1. Feasibility assessment

What is feasible now

- Detect the active executable/window, identify a known profile, and ask the user to confirm the target.
- Rank legitimate interfaces: official API/SDK first, official mod/accessibility interface second, telemetry third, screen/audio input and normal keyboard/mouse/controller last.
- Capture a selected window at interactive rates; run OCR, UI detection, template matching, object detection, optical flow, and local vision models.

- Execute closed-loop skills with bounded duration, safety constraints, and postcondition checks.
- Store session memory, per-game maps/calibrations, successful skills, and failure signatures.
- Use an MCP control plane for setup, goals, inspection, planning, high-level execution, diagnostics, and audit.

What is not universally feasible

- Reliable zero-shot completion of arbitrary unseen 3D games.
- Exact state recovery from pixels when state is hidden, occluded, ambiguous, or off-screen.
- Human-level reaction in FPS, rhythm, racing, or flight control if every action waits for a remote LLM.
- A universal semantic action such as ATTACK without a profile defining context, bindings, hold duration, camera policy, and success signals.
- Guaranteed progress in long narrative games whose objectives require undocumented knowledge, precise mechanics, or many hours.
- Safe/legal automation of games whose terms prohibit bots or whose anti-cheat rejects injected input.
- Inferring that an action succeeded merely because input was sent.

Feasibility verdict

Buildable as a research/assistive platform: YES.

Reliable autonomous substitute across the full collection without game-specific work: NO.

Scalable architecture that progressively adds games: YES.

Appropriate for competitive/live-service automation: generally NO unless expressly permitted.

2. Core architectural decision

The minimum universal contract should be:

OBSERVE → FUSE → GET_BELIEF_STATE → SET_GOAL → DECOMPOSE → SELECT_SKILL → ACT → VERIFY → REMEMBER → RECOVER.

“State” must be a belief, not a fully known truth. Every field carries value, confidence, timestamp, source, and freshness. Example:

```
player.health = {value: 72, confidence: 0.93, source: "hud_ocr", age_ms: 84}
vehicle.speed_mps = {value: 18.4, confidence: 1.0, source: "beamngpy.electrics", age_ms: 10}
objective.text = {value: "Reach the hangar", confidence: 0.78, source: "ocr", age_ms: 420}
enemy.count = {value: 2, confidence: 0.61, source: "detector+tracker", age_ms: 35}
```

Unknown is a valid first-class value. Planning must distinguish absent, stale, uncertain, contradictory, and unsupported information.

- Goal manager: converts user intent into success criteria, constraints, stop conditions, and intervention rules.
- Hierarchical planner: mission plan → subgoals → skill selection.
- Memory service: episodic, semantic, spatial, procedural, and failure memory.
- Verification/recovery manager: postconditions, divergence, retries, alternatives, safe stop.
- MCP server: typed inspection and high-level control; never the frame-rate control loop.

Data plane

- Capture workers: Windows Graphics Capture/D3D, optional audio capture, structured adapters.
- Perception graph: crop → preprocess → detect/OCR/track → scene/UI model.
- Real-time state estimator: merges telemetry and perception into the belief state.
- Skill runtime: finite-state machines, behavior trees, trajectory generators, PID/MPC controllers, local policies.
- Input backends: game API, mod bridge, keyboard/mouse, or controller.
- Recorder/evaluator: timestamped observations, actions, outcomes, metrics —subject to privacy policy.

Isolation boundary

The planner cannot directly emit arbitrary OS input. It chooses from profile-approved semantic actions or skills. The action gateway checks active window, game identity, mode, rate limits, key-up cleanup, coordinate bounds, online policy, and emergency stop before dispatch.

4. Interface selection policy

Score each candidate interface on observability, controllability, latency, reliability, legitimacy, maintenance cost, and invasiveness. Select the strongest permitted combination, not merely the richest technically possible interface.

Priority order:

1. Official game API/SDK with readback and control.
2. Official mod/scripting interface in a user-approved local/modded environment.
3. Documented telemetry/output protocol plus ordinary input.
4. Accessibility/UI Automation for launchers and menus where exposed.
5. Window capture + local perception + ordinary keyboard/mouse/controller.
6. Human checkpoint when state/action cannot be established safely.

Explicitly excluded: anti-cheat bypass, process injection, protected memory scraping, packet manipulation, hidden-state extraction, driver tricks intended to evade detection, and unattended online competitive play.

5. Perception architecture

Structured perception

Research and design only — no gameplay automation implemented

Adapters should publish normalized observations while preserving raw names and units.

Visual perception

Use a staged pipeline rather than one giant vision-language prompt:

1. Capture only the selected window with monotonic timestamp and frame ID.
2. Detect scene cuts, loading screens, menus, gameplay, pause, dialogue, death, and disconnection.
3. Apply profile ROIs for health, minimap, objectives, ammo, rhythm lanes, cockpit instruments, and prompts.
4. Run cheap detectors every frame or every few frames: color masks, templates, feature matching, OCR, optical flow.
5. Run object detection/segmentation at a lower cadence.
6. Invoke a VLM only for ambiguity, novel screens, semantic descriptions, puzzle interpretation, or profile bootstrapping.
7. Track objects and UI across frames; never reason from an isolated screenshot if motion matters.

OCR should return text boxes and confidence. Tesseract is suitable for stable high-contrast UI; scene text may need PaddleOCR or a compact text-recognition model. Profile-specific preprocessing—scale, threshold, color channel, sharpening, and font region—is more important than choosing one universal OCR engine.

Audio perception

Audio is useful for rhythm onset/beat tracking, damage alarms, dialogue completion, engine state, proximity cues, and menu confirmation. It should be optional and local. Use event detectors and spectral/onset features, not speech recording by default. Never capture voice calls or other people without consent.

Category-specific perception

Driving: lane/road geometry, optical flow, minimap, speed; structured telemetry when available.

Flight: SimVars/instruments, attitude/airspeed/altitude; visual runway and cockpit state.

FPS/action: object detector + tracking + depth/motion cues; high-frequency local policy.

Horror: scene/UI state, threat/audio cues, map/route memory, conservative recovery.

Puzzle-platform: geometry, player pose, platforms, affordances, checkpoint recognition.

Rhythm: audio onset/beat phase plus lane/note detector; sub-20–40 ms local timing path.

Strategy/sandbox: UI layout, OCR, cursor state, map segmentation; slower planner acceptable.

Open world: objective OCR, minimap localization, route memory, reusable navigation skills.

6. State representation

Use a typed GameBeliefState with namespaces:

- session: game, build, profile, mode, window, online status, paused/loading/focused.
- player: pose, health, status, abilities, inventory, equipment.
- vehicle/aircraft: pose, velocity, controls, systems, damage.
- world: map/local geometry, entities, hazards, interactables, time/weather.
- UI: screen type, dialogs, prompts, menus, selected item, objective text.

Keep the raw observation log separate from current state. State fusion uses source priority, freshness windows, confidence calibration, and contradiction alerts. A screenshot-derived speed estimate must not overwrite fresh SimConnect speed.

7. Planning architecture

One planner is insufficient. Use four levels:

Strategic planner (seconds to minutes)

Language model interprets the user goal, constraints, game knowledge, and current belief. Produces an objective graph and selects skills. It is called on goal change, major state transition, uncertainty, or failure—not every frame.

Tactical planner (100–1000 ms)

Behavior tree or finite-state controller selects local tactics: approach, evade, interact, search, align, brake, heal. It consumes structured state and can switch branches quickly.

Skill controller (10–100 ms)

Game/category-specific closed-loop skill: lane following, aiming, jumping, menu navigation, rhythm tapping, altitude hold. Use PID/MPC, learned policies, trajectory playback with feedback, or deterministic FSMs.

Safety reflex (1–20 ms where possible)

Release stuck keys, emergency brake/pause, prevent focus leakage, abort on lost capture, cap controller axes, stop after user input/emergency hotkey.

The planner should output declarative skill calls with preconditions, parameters, expected effects, timeout, verification predicate, and recovery choices.

8. Action abstraction

SemanticAction is not merely a name. It contains:

- target profile and context (gameplay/menu/vehicle/on-foot)
- backend and binding
- press/hold/release or analog trajectory
- duration/rate limits
- preconditions
- predicted state delta
- postcondition and verification window
- interruption and cleanup behavior

Examples: MOVE(direction, intensity, duration), LOOK(delta/rate), INTERACT(target_hint), SELECT(menu_item), DRIVE(target_speed, curvature), FLY(target_heading, altitude, airspeed), ATTACK(mode, target_track), WAIT_FOR(predicate), PAUSE_SAFE.

For Windows keyboard/mouse, SendInput is the legitimate baseline. Microsoft documents that it injects an ordered INPUT sequence and may be blocked; sending input is not proof of reception. For gamepads, prefer a supported virtual-controller solution or a physical programmable controller only after current driver support and game rules are verified.

Every action must guarantee release cleanup. Focus must be verified immediately before dispatch. Any human input can trigger cooperative takeover and cancel the current trajectory.

9. Verification and recovery

Each skill is a transaction-like loop:

PRECONDITION → ACT → OBSERVE UNTIL DEADLINE → EVALUATE POSTCONDITION → COMMIT MEMORY OR RECOVER.

Verification can be direct telemetry, UI text/state, object displacement, animation/state transition, audio cue, or multiple independent signals. Define negative evidence and timeout explicitly.

Failure classifier:

- action not delivered: focus/UIPI/binding/controller problem
- action delivered but ineffective: cooldown, wrong context, blocked movement
- perception stale/wrong: capture loss, scene transition, OCR/detector error
- plan wrong: invalid assumption or unavailable prerequisite
- dynamics failure: overshoot, collision, instability
- policy/legitimacy stop: online mode or anti-cheat boundary

Recovery uses an escalation ladder: retry once with changed timing → re-observe and recalibrate → choose alternate skill → backtrack/reset checkpoint → ask user. A failed action fingerprint prevents blind repetition.

10. Memory architecture

Working memory

Last seconds of observations/actions, trackers, pending postconditions, current tactical state. Ring buffer; discarded after session unless debugging is authorized.

Episodic memory

Compact event records: state summary, action, result, failure cause, screenshot references. Session-scoped by default.

Semantic game memory

Controls, mechanics, objective types, UI vocabulary, verified interface capabilities. Versioned by game build/profile.

Spatial memory

Topological graph, landmarks, routes, hazards, checkpoint relationships; coordinates only when a structured interface supports them.

Procedural memory

Reusable skills with preconditions, parameters, success rate, latency distribution, known failure contexts, and version compatibility.

Safety/privacy rules

Do not persist full continuous video by default. Redact account names/chat. Separate profiles per user. Never transfer memory between games unless the schema explicitly maps a generic concept. Promote a learned fact to persistent memory only after repeated verification or user confirmation.

11. Dynamic tool discovery

Create a signed/curated capability catalog, not an unrestricted web downloader. Each tool record includes source, version, license, OS, install method, hash/signature, capabilities, permissions, network behavior, health check, and uninstall path.

Discovery sequence: inspect game profile → probe local interfaces/tools → consult official game/engine documentation → query curated catalog → propose installation plan → explicit approval → install in managed cache → health test → register. Drivers, mods, and game-file changes require stronger approval and rollback instructions. Never auto-install memory editors, injectors, anti-cheat bypasses, or unsigned input drivers.

Likely shared tools: Windows Graphics Capture/D3D capture; OpenCV; ONNX Runtime/DirectML or CUDA providers; OCR; audio onset analysis; SendInput wrapper; supported controller backend; SQLite/PostgreSQL plus vector retrieval; Prometheus/OpenTelemetry-style metrics; ffmpeg only for offline trace handling. Game-specific SDKs remain adapters.

12. MCP architecture

MCP is the supervisory interface, not the real-time transport. Do not send every frame through MCP or wait for a tool call per keypress.

Recommended read tools:

- game_detect, game_list_windows, game_get_profile, capability_discover
- interface_probe, perception_get_state, perception_get_frame_summary
- memory_query, skill_list, action_list, agent_status, metrics_get
- plan_preview, verification_get, trace_get

Recommended consequential tools:

- session_create/close
- profile_create/update/calibrate
- goal_set/cancel
- agent_start/pause/resume/stop
- skill_execute
- tool_install_plan/approve/install
- memory_promote/delete

State-changing tools should show game/window, online policy, interface, action scope, duration, expected effects, verification, and emergency-stop binding. Starting autonomous input always requires explicit approval. MCP responses return job/session IDs; the runtime continues independently and exposes events/status.

A SKILL.md is valuable. It should force the agent to detect and confirm the game, rank interfaces, check online policy, calibrate before acting, preview the plan, obtain approval, use temporally extended skills, verify every subgoal, escalate failures, and stop on uncertainty. It should prohibit raw unbounded input, anti-cheat bypass, hidden-state

- OCR/UI detector: 2–15 Hz or event-triggered
- object detector: 5–30 Hz
- VLM scene interpretation: 0.2–2 Hz, event-triggered
- tactical planning: 2–10 Hz
- strategic LLM: transitions/failures, often below 0.2 Hz

Budget by timestamp, not assumed cadence. Discard stale inference. Use bounded queues of size one or two; newest observation wins for real-time control. A result older than its skill's freshness threshold cannot trigger an action.

Current computer-use research reports planning/reflection dominating latency and shows discrete screenshot agents failing moving targets because action is based on stale state. This directly argues for local trackers and controllers beneath the LLM.

14. Weak-model compatibility

Move competence out of prompts:

- expose normalized typed state, not raw pixels alone
- provide profile-specific action menus
- compile strategies into behavior trees/skills
- maintain objective and failure graphs
- generate concise change summaries between states
- retrieve only relevant mechanics and prior episodes
- enforce pre/postconditions in code
- provide deterministic menu navigation and control loops
- return ranked recovery choices

The model decides “which verified skill next,” not “which scan code for 17 ms.” A weaker model can succeed if perception and skills sharply constrain its action space.

15. Game collection interface matrix

BeamNG.drive

Best mode: API/telemetry hybrid where BeamNGpy compatibility/licensing is available; otherwise screen + controller. BeamNGpy documents vehicle control, synchronized state, cameras, semantic annotations, lidar/radar, roads, damage, electrics, IMU, powertrain, deterministic stepping, scenario and recovery controls. This is the best research benchmark in the collection. Caveat: validate exact BeamNG.drive versus BeamNG.tech feature availability and installed version.

Microsoft Flight Simulator

Best mode: SimConnect + visual runway/cockpit verification. SimConnect is an official out-of-process client API for simulation data, events, flight monitoring, hardware, cameras, mission extensions, and AI objects. Aircraft-specific controls often need dedicated profiles, Input Events, LVars/HVars or vendor SDKs. Never guess cockpit commands across aircraft. High-level flight phases require deterministic controllers and strict envelopes.

WorldBox God Simulator

Best mode: screen/UI for baseline. Community/native mod systems may expose Unity state but are not equivalent to a stable official automation API; require version-specific consent and maintenance. Strategy timescales are forgiving, making this a good screen-mode benchmark.

Plague Inc

Best mode: pure screen/UI + OCR + mouse. Turn/pause-friendly strategic decisions and stable panels make it suitable early, but UI layouts and hidden mechanics need a profile/knowledge base.

Melatonin

Best mode: screen + audio with a local onset/phase detector and calibrated key timing. LLM only selects level/strategy. Remote reasoning is far too slow for note timing. This is a useful latency benchmark after simpler games.

No Man's Sky

Best mode: hybrid screen/controller and profile knowledge in offline/single-player contexts. No stable official public gameplay API was verified. Community modding is version-sensitive and should not become a hidden-state shortcut. Navigation, inventory, flight and on-foot modes need separate skills.

Genshin Impact

Recommendation: do not automate ordinary live-service gameplay. Treat as disabled unless current official rules expressly permit the exact use. No process inspection, injection, packet access, anti-cheat evasion, or farming bot. Accessibility research should use a separate permitted/offline test environment, not the live account.

Solo Leveling ARISE OVERDRIVE

Treat online/live-service status and current terms as a hard policy gate. Default to no autonomous gameplay until permission is verified. Screen-level assistance is not automatically permitted automation.

Choo-Choo Charles; Dark Deception; Super Dark Deception

Likely mode: screen/audio + keyboard/controller. Horror pursuit demands fast threat tracking, route memory and recovery. Super Dark Deception's presentation may permit more stable tile/maze extraction; validate experimentally. No official structured control interface was verified.

FNaF: Security Breach; MiSide; Little Nightmares

Likely mode: screen/audio + profile-specific state machines. Dialogue/cutscene/loading recognition and checkpoint recovery are central. Little Nightmares needs precise platforming/local controller; FNaF needs threat/UI/audio fusion. No verified official telemetry APIs.

God of War 2018; Marvel's Spider-Man: Miles Morales; Transformers: The Game

Likely mode: screen + controller. Combat and camera control require fast learned/profile-specific policies; accessibility options can reduce difficulty but are not structured automation APIs. Start with navigation/menu subgoals, not full combat autonomy.

Beam-like action titles: Dead As Disco; PRAGMATA; InfinitePizza

Interfaces and even exact installed builds must be probed locally; no structured API should be assumed. Rhythm/action mechanics imply local timing loops. PRAGMATA requires fresh post-release/build-specific investigation before profile claims.

16. Shared versus game-specific components

Shared: session/capability management, capture, timestamping, OCR framework, model serving, belief-state envelope, planner contracts, action gateway, approvals, verification engine, memory stores, metrics, traces, emergency stop, MCP.

Category-specific: driving controller, flight controller, FPS tracker/aim policy, platforming controller, rhythm synchronizer, strategy UI planner, open-world navigator.

Game-specific: executable identity/build, rules/permission, window/capture calibration, controls, UI ROIs, telemetry names, state mappings, mission semantics, detectors/templates, action postconditions, recovery/checkpoints, knowledge, tests.

Game profiles enhance rather than replace universality. They are adapters and declarative configuration around a shared runtime.

17. Hardest technical problems

1. Reliable perception under motion blur, effects, camera movement, occlusion and UI changes.
2. Closed-loop real-time control while slow models reason asynchronously.
3. Grounding abstract goals in game-specific mechanics and long horizons.
4. Verification when success is subtle, delayed, or hidden.
5. Learning without corrupting memory from false perceptions.
6. Generalizing actions across on-foot, vehicle, menus and cutscenes.
7. Camera control and 3D localization from monocular pixels.
8. Safe control handoff, focus loss and stuck-input prevention.
9. Maintaining profiles across game updates/resolutions/localizations.
10. Policy compliance for online games and third-party tools.

18. Recommended proof of concept

Phase 0 — Evaluation harness

Build a synthetic game/window with known ground truth, deterministic menus, moving targets, driving lane and failure injection. Measure capture-to-action latency, stale actions, verification accuracy, stuck-key cleanup, and recovery.

Phase 1 — BeamNG structured driving

Use an authorized BeamNGpy-compatible setup. Goal: follow waypoints, maintain speed/lane, stop at destination, recover after forced deviation. Compare telemetry-only, vision-only and hybrid. This validates state fusion and real-time skills

Plague Inc or WorldBox. Goal: navigate menus and complete a bounded scenario. This validates OCR, UI grounding, profile calibration, planning and persistent mechanics memory.

Phase 3 — Offline 3D navigation

Choose one title with repeatable early checkpoint and no anti-cheat. Goal: move between landmarks and interact, not complete the whole campaign. Add local navigation/camera skill and checkpoint recovery.

Phase 4 — Rhythm benchmark

Melatonin with permitted offline play. Build audio/visual phase estimator and local key scheduler. LLM only chooses strategy.

Exit criteria for each phase

At least 30 seeded trials; report success rate, median/p95 action latency, false-action rate, recovery success, interventions, stale-frame drops, crashes, and profile portability across resolution changes. Record failures, not only successful demonstrations.

19. Scaling plan

Profiles are versioned packages with manifest, policy, interface probes, state schema mapping, actions, skills, perception graph, calibrations, recovery, tests and compatibility. A profile registry can distribute metadata, but model weights/mods remain separately reviewed.

Add games by leverage:

1. Structured APIs: BeamNG, MSFS.
2. Slow screen games: Plague Inc, WorldBox.
3. Constrained navigation/puzzle games.
4. Open-world/vehicle hybrids.
5. High-speed combat and rhythm last.
6. Online prohibited/unclear titles remain disabled.

A new profile is not “supported” until its end-to-end signal path, actions, verification and recovery pass a repeatable test suite on the exact build.

20. Final recommendation

Proceed, but frame the project as a universal game-agent runtime with progressively validated game profiles—not a universal autonomous player delivered in one step. Make BeamNGpy and SimConnect flagship structured adapters, screen mode a disciplined perception stack, and skills the unit of execution. Keep LLM calls above the real-time loop. Build verification before ambitious planning. Treat policy and online status as runtime capabilities that can disable action.

The architecture can eventually cover much of the collection at varying autonomy levels. Some games may achieve full bounded-objective autonomy; others will remain assisted or experimental. That truthful capability model is essential to a system that is safe, debuggable, and useful.

Research and design only — no gameplay automation implemented

[1] BeamNGpy documentation and API:

<https://documentation.beamng.com/api/beamngpy/master/beamngpy.html> and
<https://beamngpy.readthedocs.io/>

[2] Microsoft Flight Simulator 2024 SimConnect SDK:

https://docs.flightsimulator.com/msfs2024/html/6_Programming_APIS/SimConnect/SimConnect_SDK.htm

[3] Teardown official scripting API and modding docs:

<https://teardowngame.com/modding/api.html> and <https://teardowngame.com/modding/>

[4] Microsoft SendInput/keyboard input overview:

<https://learn.microsoft.com/windows/win32/inputdev/about-keyboard-input>

[5] Windows.Graphics.Capture namespace and frame-pool architecture:

<https://learn.microsoft.com/uwp/api/windows.graphics.capture>

[6] Tesseract OCR documentation: <https://tesseractocr.org/doc/>

[7] Voyager hierarchical skill/memory architecture: <https://arxiv.org/html/2305.16291>

[8] OSWorld-Human latency analysis: <https://arxiv.org/pdf/2506.16042>

Research caveat: official sources were preferred. For games without verified official APIs, the matrix explicitly labels interfaces as unverified/likely rather than asserting absence absolutely. Terms and supported interfaces change; profiles must re-check them at runtime and on game updates.